

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 4

Roll no: 62 PRN No: 12411517 Name of Student: Akshada Lohakare

Subject Name : Deep Learning

Problem Statement : . Develop an LSTM-based model for time-series forecasting using stock price, weather, or sales datasets.

Algorithm :

1. **Load the dataset** containing EV charging records.
2. **Select relevant features** such as energy consumed, charging power, queue length, electricity price, and vehicle type.
3. **Convert and sort the timestamp** so the data is arranged in chronological order.
4. **Handle missing values** using forward and backward filling.
5. **Encode vehicle type** into numerical values using one-hot encoding.
6. **Separate input features and target**, where station_load is the target.
7. **Split the data chronologically** into 80% training and 20% testing data.
8. **Normalize the data** using Min-Max Scaling.
9. **Create time sequences** using the previous 24 observations as input.
10. **Build the LSTM model** with LSTM, Dropout, and Dense layers.
11. **Train the model** using the Adam optimizer and MSE loss.
12. **Predict station load** using the test sequences.
13. **Convert predictions back** to the original scale.
14. **Evaluate the model** using MAE, RMSE, and MAPE.
15. **Plot actual and predicted values** to visualize the forecasting performance.

Code :

```
#import the libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

#df = pd.read_csv("ev_charging_dataset.csv")

print("Dataset Shape:", df.shape)
print("\nColumns:")
print(df.columns)

features = [
    "timestamp",
    "vehicle_type",
    "energy_consumed_kWh",
    "charging_power_kW",
    "station_load",
    "queue_length",
    "electricity_price"
]

df = df[features]

print(df.head())

# Convert timestamp to datetime
df["timestamp"] = pd.to_datetime(df["timestamp"])

# Sort according to time
df = df.sort_values("timestamp")

# Check missing values
print("\nMissing Values:")
```

```
print(df.isnull().sum())

# Remove missing values
df = df.dropna()

print("\nDataset after preprocessing:")
print(df.head())


#encoding
df = pd.get_dummies(
    df,
    columns=["vehicle_type"],
    dtype=int
)

print("\nColumns after encoding:")
print(df.columns)


vehicle_columns = [
    col for col in df.columns
    if col.startswith("vehicle_type_")
]

feature_columns = [
    "energy_consumed_kWh",
    "charging_power_kW",
    "queue_length",
    "electricity_price"
]

feature_columns += vehicle_columns

target_column = "station_load"

X = df[feature_columns]
y = df[target_column]

print("\nInput Features:")
print(feature_columns)

print("\nTarget:")
```

```
print(target_column)

print("\nX Shape:", X.shape)
print("Y Shape:", y.shape)


# train-test and split
split = int(len(df) * 0.8)

X_train = X.iloc[:split]
X_test = X.iloc[split:]

y_train = y.iloc[:split]
y_test = y.iloc[split:]

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))


# scaling the data
from sklearn.preprocessing import MinMaxScaler

# Create scalers
feature_scaler = MinMaxScaler()
target_scaler = MinMaxScaler()

# Scale input features
X_train_scaled = feature_scaler.fit_transform(X_train)
X_test_scaled = feature_scaler.transform(X_test)

# Scale target
y_train_scaled = target_scaler.fit_transform(
    y_train.values.reshape(-1, 1)
)

y_test_scaled = target_scaler.transform(
    y_test.values.reshape(-1, 1)
)

print("X_train_scaled shape:", X_train_scaled.shape)
print("y_train_scaled shape:", y_train_scaled.shape)
```

```
# create time series sequence
import numpy as np

def create_sequences(X, y, time_steps=24):

    X_sequences = []
    y_sequences = []

    for i in range(time_steps, len(X)):

        X_sequences.append(
            X[i-time_steps:i]
        )

        y_sequences.append(
            y[i]
        )

    return np.array(X_sequences), np.array(y_sequences)

time_steps = 24

X_train_seq, y_train_seq = create_sequences(
    X_train_scaled,
    y_train_scaled,
    time_steps
)

X_test_seq, y_test_seq = create_sequences(
    X_test_scaled,
    y_test_scaled,
    time_steps
)

print("X Train Shape:", X_train_seq.shape)
print("Y Train Shape:", y_train_seq.shape)

print("X Test Shape:", X_test_seq.shape)
print("Y Test Shape:", y_test_seq.shape)
```

```

# building lstm model
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

model = Sequential()

model.add(
    LSTM(
        64,
        return_sequences=True,
        input_shape=(
            X_train_seq.shape[1],
            X_train_seq.shape[2]
        )
    )
)

model.add(Dropout(0.2))

model.add(
    LSTM(32)
)

model.add(Dropout(0.2))

model.add(
    Dense(16, activation="relu")
)

model.add(
    Dense(1)
)

model.summary()

#model compilation
model.compile(
    optimizer="adam",
    loss="mse"
)

```

```
print("Model compiled successfully.")

#model config
history = model.fit(
    X_train_seq,
    y_train_seq,
    epochs=30,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)

#visualization
# plots
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("LSTM Training and Validation Loss")

plt.legend()

plt.show()

#prediction vs actual plot visualization
```

```
predictions = model.predict(X_test_seq)

print("Prediction shape:", predictions.shape)

predictions_actual = target_scaler.inverse_transform(
    predictions
)

y_test_actual = target_scaler.inverse_transform(
    y_test_seq
)

print("First 5 actual values:")
print(y_test_actual[:5])

print("\nFirst 5 predicted values:")
print(predictions_actual[:5])


plt.figure(figsize=(12, 6))

plt.plot(
    y_test_actual,
    label="Actual Station Load"
)

plt.plot(
    predictions_actual,
    label="Predicted Station Load"
)

plt.xlabel("Time Steps")
plt.ylabel("Station Load (kW)")

plt.title(
    "Actual vs Predicted EV Charging Station Load"
)

plt.legend()

plt.show()
```


Output :

```
Dataset Shape: (8354, 7)
```

```
Columns:
```

```
Index(['timestamp', 'vehicle_type', 'energy_consumed_kWh', 'charging_power_kW',  
      'queue_length', 'station_load', 'electricity_price'],  
      dtype='object')
```

```
      timestamp vehicle_type energy_consumed_kWh charging_power_kW \  
0  2025-01-01 00:00:00   Two-Wheeler          30.528616           7  
1  2025-01-01 00:15:00   Two-Wheeler          41.504507          50  
2  2025-01-01 00:30:00           Car          45.016306          50  
3  2025-01-01 00:45:00   Two-Wheeler          28.291670          11  
4  2025-01-01 01:00:00   Two-Wheeler          55.810835          11
```

```
      station_load queue_length electricity_price  
0      15.161672           4           13.66  
1      20.997219           3            5.47  
2      31.606151           8            9.50  
3      21.803050           3            6.22  
4      15.626266           5           13.42
```

```
Missing Values:
```

```
timestamp           0  
vehicle_type        0  
energy_consumed_kWh 0  
charging_power_kW    0  
station_load         0  
queue_length         0  
electricity_price    0  
dtype: int64
```

```
Dataset after preprocessing:
```

```
      timestamp vehicle_type energy_consumed_kWh charging_power_kW \  
0  2025-01-01 00:00:00   Two-Wheeler          30.528616           7  
1  2025-01-01 00:15:00   Two-Wheeler          41.504507          50  
2  2025-01-01 00:30:00           Car          45.016306          50  
3  2025-01-01 00:45:00   Two-Wheeler          28.291670          11  
4  2025-01-01 01:00:00   Two-Wheeler          55.810835          11
```

```
      station_load queue_length electricity_price  
0      15.161672           4           13.66  
1      20.997219           3            5.47  
2      31.606151           8            9.50  
3      21.803050           3            6.22  
4      15.626266           5           13.42
```

```
Input Features:
```

```
['energy_consumed_kWh', 'charging_power_kW', 'queue_length', 'electricity_price', 'vehicle_type_Bus', 'vehicle_type_Car', 'vehicle_type_Two-Wheeler']
```

```
Target:
```

```
station_load
```

```
X Shape: (8354, 7)
```

```
Y Shape: (8354,)
```

```
X Train Shape: (6659, 24, 7)
```

```
Y Train Shape: (6659, 1)
```

```
X Test Shape: (1647, 24, 7)
```

```
Y Test Shape: (1647, 1)
```

```
17. # building lstm model
```

Model: "sequential"

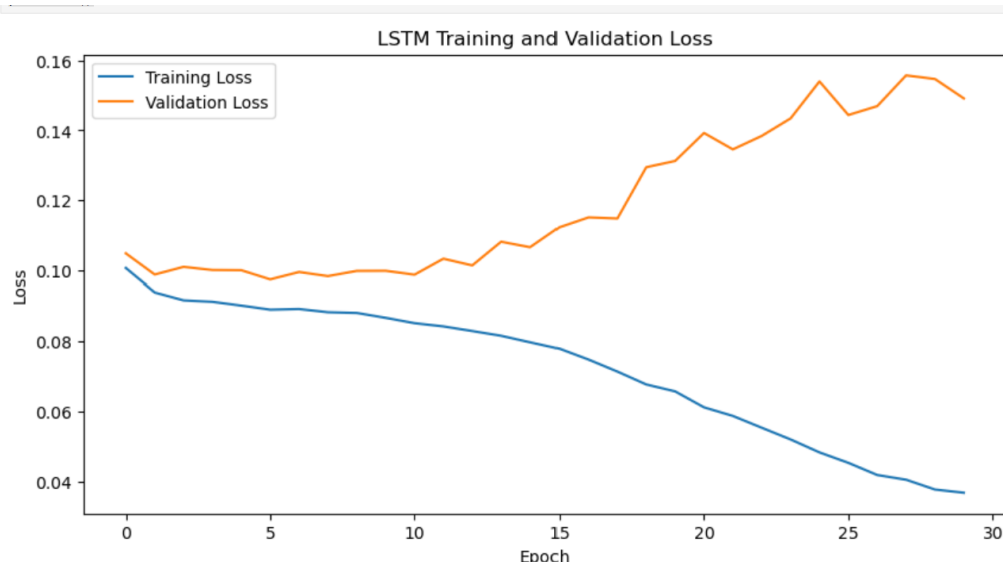
Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 24, 64)	18,432
dropout (Dropout)	(None, 24, 64)	0
lstm_1 (LSTM)	(None, 32)	12,416
dropout_1 (Dropout)	(None, 32)	0
dense (Dense)	(None, 16)	528
dense_1 (Dense)	(None, 1)	17

Total params: 31,393 (122.63 KB)

Trainable params: 31,393 (122.63 KB)

```
Epoch 1/30
188/188 ————— 13s 28ms/step - loss: 0.1007 - val_loss: 0.1049
Epoch 2/30
188/188 ————— 9s 20ms/step - loss: 0.0939 - val_loss: 0.0989
Epoch 3/30
188/188 ————— 4s 20ms/step - loss: 0.0917 - val_loss: 0.1011
Epoch 4/30
188/188 ————— 4s 19ms/step - loss: 0.0913 - val_loss: 0.1001
Epoch 5/30
188/188 ————— 5s 19ms/step - loss: 0.0902 - val_loss: 0.1001
Epoch 6/30
188/188 ————— 4s 21ms/step - loss: 0.0890 - val_loss: 0.0975
Epoch 7/30
188/188 ————— 4s 21ms/step - loss: 0.0892 - val_loss: 0.0996
Epoch 8/30
188/188 ————— 5s 20ms/step - loss: 0.0883 - val_loss: 0.0984
Epoch 9/30
188/188 ————— 5s 22ms/step - loss: 0.0881 - val_loss: 0.0999
Epoch 10/30
188/188 ————— 5s 21ms/step - loss: 0.0867 - val_loss: 0.0999
Epoch 11/30
188/188 ————— 4s 20ms/step - loss: 0.0852 - val_loss: 0.0988
Epoch 12/30
188/188 ————— 4s 19ms/step - loss: 0.0843 - val_loss: 0.1034
Epoch 13/30
188/188 ————— 3s 18ms/step - loss: 0.0829 - val_loss: 0.1015
Epoch 14/30
188/188 ————— 4s 21ms/step - loss: 0.0816 - val_loss: 0.1082
Epoch 15/30
188/188 ————— 5s 19ms/step - loss: 0.0797 - val_loss: 0.1067
Epoch 16/30
188/188 ————— 6s 21ms/step - loss: 0.0779 - val_loss: 0.1123
Epoch 17/30
188/188 ————— 4s 21ms/step - loss: 0.0748 - val_loss: 0.1152
Epoch 18/30
188/188 ————— 4s 20ms/step - loss: 0.0713 - val_loss: 0.1149
Epoch 19/30
188/188 ————— 4s 20ms/step - loss: 0.0677 - val_loss: 0.1295
Epoch 20/30
188/188 ————— 4s 20ms/step - loss: 0.0657 - val_loss: 0.1313
Epoch 21/30
188/188 ————— 4s 20ms/step - loss: 0.0611 - val_loss: 0.1393
Epoch 22/30
188/188 ————— 4s 21ms/step - loss: 0.0587 - val_loss: 0.1346
Epoch 23/30
188/188 ————— 5s 20ms/step - loss: 0.0553 - val_loss: 0.1384
Epoch 24/30
188/188 ————— 4s 20ms/step - loss: 0.0520 - val_loss: 0.1434
Epoch 25/30
188/188 ————— 4s 21ms/step - loss: 0.0483 - val_loss: 0.1540
Epoch 26/30
188/188 ————— 4s 22ms/step - loss: 0.0453 - val_loss: 0.1444
Epoch 27/30
```

- **LSTM Training and Validation visualization:**



- **Actual vs Predicted EV Charging Station Load visualization:**

